# Paradigmi di programmazione

# Contents

**1 Introduzione** **6**

**2 Descrizione dei paradigmi** **7**
2.1 Paradigma imperativo . . . 7
2.2 Paradigma object-oriented . . . 7
2.3 Paradigma funzionale . . . 7
2.4 Programmazione concorrente . . . 7
2.5 Programmazione parallela . . . 7

**3 Il ruolo chiave dei tipi** **8**

**4 Linguaggi** **9**
4.1 Fortran . . . 9
4.2 ALGOL 60 . . . 9
4.3 LISP . . . 9
4.4 SIMULA 67 . . . 9
4.5 C . . . 10
4.6 Pascal . . . 10
4.7 Smalltalk . . . 10
4.8 PROLOG . . . 10
4.9 C++ . . . 10
4.10 Java . . . 10
4.11 Altri linguaggi popolari . . . 11
4.12 Come crescono i linguaggi? . . . 11

**5 Approccio scientifico** **12**

**6 Lambda Calculus** **13**
6.1 Definizione . . . 13
6.2 Lambda espressioni . . . 13
6.2.1 Segnatura e funzione identità . . . 13
6.2.2 Applicazione . . . 13
6.2.3 Funzione costante . . . 13
6.2.4 Funzioni annidate . . . 14
6.2.5 Ordine di semplificazione . . . 14
6.2.6 Funzioni come parametro . . . 14
6.2.7 Prolungamento di espressione . . . 14
6.2.8 Parametri attuali numerici . . . 15
6.2.9 Ambiguità . . . 15
6.3 Convenzioni sintattiche . . . 15
6.4 Variabili libere e legate . . . 15
6.5 Definizione sintattica . . . 16
6.6 Alberi per lambda espressioni . . . 16
6.7 Alpha equivalenza . . . 16
6.8 Valutazione di lambda espressioni . . . 16
6.9 Beta reduction e beta equivalenza . . . 17

6.10 Non terminazione . . . 18
6.11 Forma normale . . . 18
6.12 Teorema di Church Rosser . . . 18
6.13 Funzioni di ordine superiore e currying . . . 18
6.14 Valutazione dell'applicazione di funzione . . . 19
6.15 Implementazione della ricorsione . . . 19
6.16 Variabili booleane . . . 20
6.17 Condizionale . . . 21
6.18 Numeri . . . 21

**7 Regole di inferenza** **22**
7.1 Sintassi . . . 22
7.2 Successore . . . 22
7.3 Predecessore . . . 22
7.4 IsZero . . . 22
7.5 If-Else . . . 22

**8 Controllo dei tipi** **23**
8.1 Tipi . . . 23
8.2 Well-typed e ill-typed . . . 23
8.3 Type Safety: Correttezza . . . 23
8.4 Controllo su successore . . . 23
8.5 Controllo su predecessore . . . 23
8.6 Controllo su isZero . . . 23
8.7 Controllo su if-else . . . 24

**9 Lambda Calculus Tipato** **25**
9.1 L'introduzione dei tipi . . . 25
9.2 Sistema di controllo dei tipi . . . 25
9.3 Sintassi dei tipi . . . 26
9.4 Ambiente di tipo . . . 26
9.5 Substitution Lemma . . . 27
9.6 Dichiarazioni locali . . . 27
9.7 Ricorsione . . . 27

**10 OCaml** **28**
10.1 Antenati . . . 28
10.2 Come mai NON studiare OCaml (Ma invece si) . . . 28
10.3 Esecutore di OCaml . . . 28
10.4 Overloading degli operatori e cast . . . 29
10.5 Tipi di OCaml . . . 29
10.5.1 Tipi di base . . . 29
10.5.2 Tipi complessi . . . 29
10.5.3 Tipi custom . . . 30
10.6 Operazioni logiche . . . 30
10.7 Operazioni sulle stringhe . . . 30
10.8 Funzioni e lambda calcolo . . . 31
10.8.1 Applicazione parziale . . . 31
10.8.2 Encoding . . . 31

10.9 Scope . . . 32
10.10Costrutti . . . 32
10.11Ricorsione . . . 32
10.12Sum if true . . . 32
10.13Type inference e polimorfismo . . . 33
10.14Liste . . . 33
10.14.1 Funzioni ricorsive su liste . . . 33
10.14.2 Funzioni higher-order su liste . . . 34
10.14.3 Accumulatore . . . 34
10.14.4 Filter . . . 35
10.14.5 Map . . . 35
10.14.6 Reduce (fold) . . . 35
10.15Pattern matching . . . 35
10.16Wildcard . . . 35
10.17Record . . . 36
10.17.1 Dot notation . . . 36
10.17.2 Copia di record . . . 36
10.17.3 immutabilità . . . 36
10.18Variant . . . 36
10.18.1 Enum . . . 36
10.18.2 Numeri naturali estesi . . . 37
10.19Tipi ricorsivi . . . 37
10.19.1 Alberi . . . 37
10.19.2 Abstract Syntax Tree . . . 38
10.20Programmazione imperativa (introduzione) . . . 38
10.20.1 Nuova sintassi degli array . . . 38
10.20.2 Record mutabili . . . 39
10.20.3 Riferimenti (refs) . . . 39
10.20.4 Sequenze di espressioni e cicli . . . 39
10.20.5 Eccezioni . . . 40
10.21Encoding da lambda calcolo a OCaml . . . 40

**11 Macchine astratte e interpreti 41**
11.1 Macchine astratte . . . 41
11.1.1 Interprete . . . 42
11.1.2 Componente di controllo . . . 42
11.1.3 Memoria . . . 42
11.1.4 Operazioni primitive . . . 42
11.2 Run-Time Support . . . 43
11.3 Compilatore . . . 43
11.3.1 Struttura del compilatore . . . 43

**12 Sviluppo di interpreti 45**
12.1 Realizzazione d'esempio . . . 45
12.1.1 Sintassi . . . 45
12.1.2 Definizione scanner e token . . . 45
12.2 Definizione parser . . . 47
12.3 Definizione interpreter . . . 49
12.3.1 Structural Operational Semantics (SOS) . . . 49

12.3.2 Realizzazione interprete big-step . . . . . 49
12.3.3 Realizzazione interprete small-step . . . . . 50
12.3.4 Dalla SOS al codice . . . . . 50
12.3.5 Interprete risultante . . . . . 51
12.4 Interprete per lamba calcolo . . . . . 51
12.4.1 Sintassi del lambda calcolo . . . . . 51
12.4.2 Semantica del lambda calcolo . . . . . 51
12.4.3 Implementazione di FV(e) . . . . . 52
12.5 Implementazione dell'interprete di un nucleo di linguaggio funzionale . . . 52

**13 Altro** **53**

**14 Esercizi** **54**
14.1 Esercizio 1 . . . . . 54
14.2 Esercizio 2 . . . . . 54
14.3 Esercizio 3 . . . . . 54
14.4 Esercizio 4 . . . . . 55
14.5 Esercizio 5 . . . . . 55
14.6 Esercizio 6 . . . . . 55

# 1 Introduzione

**Email Docente:** chiara.bodei@unipi.it

**E-Learning:** https://elearning.di.unipi.it/course/view.php?id=1167 Il Teams è privato, non si può entrare. (Future notice)

Lo scopo del corso è di facilitare l'apprendimento di nuovi linguaggi di programmazione attraverso lo studio dei paradigmi.

# 2 Descrizione dei paradigmi

Non esiste un paradigma migliore degli altri: la scelta dipende dalla situazione e dal problema da risolvere.
Molti programmi possono utilizzare, per sottoproblemi diversi, approcci differenti (**multiparadigma**).

## 2.1 Paradigma imperativo

Il **paradigma imperativo** descrive il programma come una sequenza di istruzioni che modificano lo stato del programma.
Il programmatore specifica **come** ottenere il risultato, indicando esplicitamente le operazioni da eseguire.
Esempi di costrutti tipici sono assegnamenti, cicli e istruzioni condizionali.

## 2.2 Paradigma object-oriented

Il **paradigma object-oriented** (o **a oggetti**) organizza il programma attorno a oggetti che combinano **dati** e **comportamenti**.
Gli oggetti appartengono a **classi**, che ne definiscono caratteristiche e operazioni.
I principali concetti sono **incapsulamento**, **ereditarietà**, **polimorfismo** e **astrazione**.

## 2.3 Paradigma funzionale

Il **paradigma funzionale** considera il programma come una composizione di **funzioni**, cercando di evitare modifiche dello stato e effetti collaterali.
Le funzioni possono essere trattate come valori, passate come argomenti e restituite come risultati.
Sono importanti concetti come **immutabilità**, **funzioni pure**, **ricorsione** e **funzioni di ordine superiore**.

## 2.4 Programmazione concorrente

La **programmazione concorrente** permette a più attività di essere **in esecuzione nello stesso intervallo di tempo**, anche se non vengono necessariamente eseguite simultaneamente.
Le attività possono alternarsi e comunicare o sincronizzarsi tra loro.
È utile, ad esempio, per gestire contemporaneamente richieste di utenti, operazioni di I/O o attività indipendenti.

## 2.5 Programmazione parallela

La **programmazione parallela** consiste nell'eseguire più operazioni **simultaneamente**, generalmente sfruttando più processori o core.
L'obiettivo principale è ridurre il tempo necessario per completare un'elaborazione, dividendo il problema in parti che possono essere eseguite in parallelo.
La programmazione parallela è quindi particolarmente utile per problemi scomponibili in attività indipendenti o parzialmente indipendenti.

# 3 Il ruolo chiave dei tipi

I **tipi** nella programmazione sono molto importanti, in quanto dichiarano gli intenti di dati e funzioni.
Il **Type Checking** (presente per esempio in TypeScript) assicura il corretto svolgimento delle operazioni.
Inoltre, il type checking rende il programma più efficiente ed evita errori nelle operazioni.
Si dice che il type checking offra una **correttezza parziale** (solo per alcuni tipi di errori) ed è un esempio di tecnica di analisi di programma (statica e dinamica).

# 4 Linguaggi

## 4.1 Fortran

**Fortran** (FORMula TRANsfer) è uno dei linguaggi più antichi, realizzato da **John Backus** (1924-2007) negli anni 50 all'IBM, primo linguaggio di alto livello implementato efficacemente (memoria statica, senza annidamento).
Ha introdotto le variabili come le conosciamo oggi, i cicli, le procedure, le etichette delle istruzioni e molto altro.
Le prime versioni non supportavano la ricorsione e includevano caratteristiche oggi abbandonate.

## 4.2 ALGOL 60

**ALGOL** (ALGOrithmic Language) fu progettato nel 1958-1960 come linguaggio universale.
Ha introdotto innovazioni fondamentali:

- Sintassi formalmente specificata (BNF)
- Scoping lessicale / blocchi
- Procedure modulari
- Passaggi per nome
- Procedure ricorsive
- Dichiarazioni di tipi di variabili
- Allocazione dello stack e record di attivazione

## 4.3 LISP

Il **LISP** (LISt Processor) è un linguaggio funzionale, sviluppato da **John McCarty** nel 1958 al MIT per la manipolazione di liste.
Precursore di molti concetti, tra cui la gestione dinamica della memoria (heap e garbage collector).

## 4.4 SIMULA 67

**SIMULA 67** è stato sviluppato da **Ole-Johan** e **Kristen Nygaard**, è un estensione di ALGOL 60.
Fu il primo linguaggio orientato agli oggetti, con classi, oggetti, ereditarietà e sottoclassi.
Introdusse anche passaggi per valore e riferimento, puntatori e **co-routine** (simili ai thread).

## 4.5 C

Introdotto nei Bell Labs 1972 da **Ritchie**, il suo sviluppo è strettamente legato a **UNIX** e ai sistemi operativi.
Maggiori possibilità di accessi alle funzionalità di basso livello della macchina.
Gestione manuale della memoria (puntatori ma no garbage collection). Le funzioni non si possono annidare.
Controllo dei tipi non forte.
Linguaggio imperativo.

## 4.6 Pascal

progettato da **N. Wirth** agli inizi degli anni '70, prende il nome dal filosofo **Blaise Pascal**, inventore della **pascalina**, strumento di calcolo precursore della calcolatrice moderna.
Rappresenta una versione più semplice di ALGOL 60.
Linguaggio fortemente tipizzato, gestione dinamica della memoria e scoping statico.
Offre molti tipi di dati (anche custom) coome array, record, file e set.
Supporta le funzioni.

## 4.7 Smalltalk

Introdotto negli anni '70 da **Alan Kay** allo Xerox Palo Alto Research Center per supportare nozioni di incapsulamento e information hiding.
Tutto è un oggetto tranne le variabili; la computazione avviene tramite uno scambio di messaggi.
Utilizza il garbage collector ed è dotato di GUI e IDE integrati.

## 4.8 PROLOG

**PROLOG** (PROgrammation en LOGique) fu introdotto da **Robert Kowalski** (teoria), **Marten Van Emdem** (dimostrazione sperimentale) e implementato finalmente da **Alain Colmerauer** all'inizio degli anni '70.
Linguaggio dichiarativo

## 4.9 C++

Introdotto nei Bell Labs nel 1986 da **Bjarne Stroustrup**.
Gli oggetti sono una generalizzazione di "struct".
Estende **C** con le classi ed un migliore sistema di tipi.
Supporta concetti complessi come l'ereditarietà multipla, modelli / generici (template) e la gestione delle eccezioni.

## 4.10 Java

Introdotto alla SUN 1991-1995 dal gruppo di **James Gosling**, completamente orientato agli oggetti.
Ha garbage collection e quindi la gestione della memoria non è esplicita; non c'è ereditarietà multipla e neanche generics.
Si possono usare thread in maniera concorrente.

Presta molto attenzione alla portabilità dato il suo aspetto ibrido tra linguaggio compilato e interpretato.

## 4.11 Altri linguaggi popolari

- **Imperativi:** Modula, Oberon, Ada
- **Funzionali:** Miranda, Haskell, ML, Caml, OCaml, Scheme, Common LISP
- **Orientati agli oggetti:** Objective-C, Eiffel, Modula-3, Self, C#, CLOS
- **Logici:** Gödel, LDL, ACL2, Isabelle, HOL
- **Scripting:** (Text processing) Perl, Python; (Web programming) JavaScript, PHP
- **Elaborazione dati e database:** Cobol, SQL, 4GL, XQuery
- **Programmazione di sistemi:** PL/I, PL/M, BLISS
- **Applicazioni specializzate:** APL, Forth, Icon, Logo, SNOBOL4, GPSS, Visual Basic
- **Concorrenti:** Concurrent Pascal, Concurrent C, C*, SR, Occam, Erlang, Obliq

## 4.12 Come crescono i linguaggi?

Ci sono diversi fattori che fanno crescere un linguaggio, eccone diversi:

- **Killer App**: un linguaggio è la scelta gettonata per sviluppare app di successo (Java, PHP, R)
- **Esclusività delle piattaforme**: alcuni linguaggi si impongono perchè vincolati a un ecosistema (Swift e iOS).
- **Velocità dell'upgrade**: evoluzione rapida che mantiene la semplicità (Python)
- **Grande slogan**: fa la differenza (Java: write once, run everywhere)
- **Slow and steady**: crescita lenta ma stabile e duratura (C)
- **Sintassi chiara**: abbassa la barriera d'ingresso (Python)
- **Job market**: orienta le scelte (Java, JavaScript, Python)
- **Community**: il contributo degli appassionati tramite numerosissime librerie (JavaScript, Python)

# 5 Approccio scientifico

Per studiare i linguaggi di programmazione si attuerà un **metodo scientifico**:

1. **Fatti:** la realtà che va studiata (osservazioni, esperimenti, dati...)
2. **Modello:** la rappresentazione tramite leggi matematiche.
3. **Ragionamento:** analisi della realtà attraverso il modello.

Questo metodo viene applicato in tutti i campi della scienza e ricerca e lo applicheremo anche noi in questo modo:

1. **Fatti:** linguaggi di programmazione.
2. **Modello:** semantica.
3. **Ragionamento:** comprensione e studio di proprietà.

# 6 Lambda Calculus

## 6.1 Definizione

Il **lambda calcolo** è stato introdotto nel 1935 da **Church** e può essere considerato il primo linguaggio di programmazione funzionale e setta proprio una base per studiare tale paradigma.

## 6.2 Lambda espressioni

### 6.2.1 Segnatura e funzione identità

Il lambda calculus utilizza questa segnatura:

$$\lambda x.x$$

- $\lambda$ = introduce la funzionale
- $x$ = parametro formale (1); corpo (2) la cui valutazione applicata all'input da il valore in output.
- . = separa la variabile dal corpo

La legge di corrispondenza corrisponde a:

$$f(x) = x$$

Ovvero la funzione identità.

### 6.2.2 Applicazione

Per attualizzare questa segnatura si fa cosi e si chiama **applicazione**:

$$(\lambda x.x)z = z$$

Con "z" come **parametro attuale**.

### 6.2.3 Funzione costante

Vediamo adesso un esempio di **funzione costante**:

$$\lambda x.y$$

$$(\lambda x.y)z = y$$

L'output non dipende dall'input (parametro formale).

### 6.2.4 Funzioni annidate

Addirittura, esistono annidamenti di funzioni:

$$\lambda x.\lambda y.x$$

Questo aspetta due parametri in input e restituisce sempre il primo, corrisponde a:

$$f(x, y) = x$$

$$((\lambda x.\lambda y.x)z)w = (\lambda y.z)w = z$$

La prima funzione non ritorna un valore, bensì un'espressione (o una funzione) che può essere usata a sua volta.
La parte dell'espressione che si riduce si chiama **redex**, si effettuano le sostituzioni in ordine e si osserva come si evolve per determinare un risultato.

### 6.2.5 Ordine di semplificazione

Spesso si può scegliere un ordine di semplificazione:

- Partendo dalla funzione più esterna
- Partendo dalla funzione più interna

Vediamo un esempio:

$$(\lambda x.((\lambda y.y)x))z$$

Risolvendo prima l'esterna:

$$(\lambda x.((\lambda y.y)x))z = \lambda(y.y)z = z$$

Risolvendo prima l'interna:

$$(\lambda x.((\lambda y.y)x))z = \lambda(x.x)z = z$$

### 6.2.6 Funzioni come parametro

Vediamo adesso un altro caso:

$$(\lambda f.fz)(\lambda x.x)$$

Questo usa un'intera funzione come parametro:

$$(\lambda f.fz)(\lambda x.x) = (\lambda x.x)z = z$$

### 6.2.7 Prolungamento di espressione

Le espressioni possono ricostruirsi:

$$(\lambda x.(xx))(\lambda y.y)$$

$$(\lambda x.(xx))(\lambda y.y) = (\lambda y.y)(\lambda y.y) = \lambda y.y$$

### 6.2.8 Parametri attuali numerici

Adesso vediamo un esempio con una somma:

$$((\lambda x.\lambda y.x + y)3)5$$

$$((\lambda x.\lambda y.x + y)3)5 = (\lambda y.3 + y)5 = 3 + 5 = 8$$

Basta effettuare un passaggio alla volta.

### 6.2.9 Ambiguità

Vediamo una situazione di ambiguità con 3 espressioni concatenate nella forma $(e_1, e_2, e_3)$:

$$(\lambda x.\lambda y.xy)(\lambda x.x)z$$

Per convenzione bisogna svolgere le prime due e lasciare la terza per dopo:

$$(\lambda x.\lambda y.xy)(\lambda x.x)z = (\lambda y.((\lambda x.x)y))z = \lambda(x.x)z = z$$

Questo conta anche per i parametri attuali:

$$(\lambda x.\lambda y.xy)zk$$

$$(\lambda x.\lambda y.xy)zk = (\lambda y.zy)k$$

Bisogna fare attenzione ad ambiguità tra parametri formali e attuali:

$$(\lambda x.\lambda y.xy)y$$

Se risolto, potrebbero esserci ambiguità, per questo conviene cambiare nome:

$$y = z$$

## 6.3 Convenzioni sintattiche

Per prima cosa, la **portata** (scope) del costruttore lambda si estende il più a destra possibile.

$\lambda x.\lambda y.xy$ è uguale a $\lambda x.(\lambda y.(xy))$

L'**applicazione funzionale** associa a **sinistra**:

$e_1e_2e_3$ diventa $(e_1e_2)e_3$

## 6.4 Variabili libere e legate

Una **variabile legate (bound)** a un $\lambda$ sono quelle che fanno parte di una lambda espressione, ovvero una dichiarazione (parametri formali).
Le **variabili libere (unbound)** invece corrispondono al contrario, come i parametri attuali, le variabili non enunciate da un $\lambda$, queste sono definite dall'insieme:

$$FV$$

Per esempio, possiamo applicarlo ad un'espressione:

$$FV(x) = \{x\}$$

$$FV((\lambda x.x)z) = \{z\}$$

Un termine si dice **chiuso** se:

$$Fv(E) = \emptyset$$

## 6.5 Definizione sintattica

- e ::= x
  - $\lambda x.e$ (function declaration)
  - e e (function call)

## 6.6 Alberi per lambda espressioni

Un altro tipo di rappresentazione sono gli alberi, fatti in questo modo:

- Il costruttore $\lambda$ è la radice.
- Sul ramo sinistro si trova il parametro formale.
- Sul ramo destro si trova il corpo, il quale può ricominciare con un $\lambda$.

## 6.7 Alpha equivalenza

Due espressioni sono dette **alpha equivalenza** se differiscono solo per i nomi delle variabili legate. Più semplicemente, si potrebbe "rinominare" qualche parametro per rendere le due espressioni identiche, questa operazione si chiama **alpha-conversione**.
Questa tecnica può essere applicata anche a problemi di ambiguità legati alla similarità tra nomi di parametri formali e attuali.
Vediamo alcuni esempi:

**Sostituzione negata, variabile legata**

$$((\lambda x.yx)w)\{x := \lambda k.kx\} = (\lambda x.yx)w$$

**Alpha equivalenza**

$$((\lambda x.yx)w)\{y := \lambda k.kx\} \equiv_\alpha ((\lambda z.yz)w)\{y := \lambda k.kx\} = (\lambda z.\lambda k.kxz)w$$

**Sostituzione semplice, diretta**

$$((\lambda x.yx)w)\{w := \lambda k.kx\} = (\lambda x.yx)(\lambda k.kx)$$

## 6.8 Valutazione di lambda espressioni

La notazione è questa per la valutazione:

$$eval(espressione)$$

Semplicemente, la valutazione corrisponde a:

- Osservare se bisogna effettuare un'alpha-conversione per risolvere ambiguità.
- Sostituire i parametri formali coi parametri attuali.
- Svolgere la funzione (equivalente di effettuare una chiamata ad una funzione, per capire).

- Valutare il risultato e restituirlo.

La notazione per la sostituzione è questa:

$$e_1\{x := e_2\}$$

Nozione alternativa:

$$e_1\{e_2/x\}$$

La sostituzione può essere distribuita:

$$(e_1e_2)\{x := e\} \equiv (e_1\{x := e\})(e_2\{x := e\})$$

Altri concetti importanti:

$$(\lambda y.e_1)\{x := e\} \equiv \lambda y.(e_1\{x := e\}), y \neq x \wedge y \notin FV(e)$$

**ATTENZIONE:** se nell'espressione con cui si sta effettuando la sostituzione (non la variabile da sostituire) contiene variabili con lo stesso nome di variabili legate bisogna prima effettuare l'alpha conversione per evitare ambiguità ed errori.
Infatti, la sostituzione si può effettuare solo sulle variabili non legate.

## 6.9 Beta reduction e beta equivalenza

La **beta reduction** consiste nella chiusura transitiva e riflessiva (stella di Finy).
Consiste semplicemente nell'applicazione dei parametri attuali per eseguire sostituzioni:

$$(\lambda x.xy)z$$

$$e_1\{x := e_2\}$$

$$(\lambda x.xy)z\{x := z\} \to zy$$

- $e_1 \to e_2$ se si ottiene $e_2$ da $e_1$ in **un passo** di riduzione.
- $e_1 \implies e_2$ se si ottiene $e_2$ da $e_1$ in **zero o più passi** di riduzione.

Con $\implies$ indichiamo la chiusura riflessiva e transitiva.
Una espressione $e_1$ è **beta-riducibile** se:

$$e_1 \implies e_2$$

Due espressioni $e_1$ ed $e_2$ sono **beta equivalenti** ($e_1 \equiv_\beta e_2$) quando:

- Sono identiche (modulo alpha-conversione)
- $e_1 \implies e_2$ oppure $e_2 \implies e_1$
- $e_1 \implies e$ e anche $e_2 \implies e$

## 6.10 Non terminazione

Durante lo svolgimento di un espressione si possono incontrare loop, questo concetto si chiama **non terminazione**.
Vediamo un esempio:

$$(\lambda x.(xx))(\lambda x.(xx))$$

Svolgendo l'espressione, si ricrea sempre.
In realtà, tali espressioni possono:

- Terminare in "alcuni casi" (facendo specifiche scelte di riduzione)
- Non terminare in "altri casi" (con scelte diverse)

## 6.11 Forma normale

Esistono delle procedure di valutazione delle espressioni.
Quando una lambda espressione non può essere più ridotta si dice che è in **forma normale beta**, ovvero quando non si può più applicare la beta riduzione.
Una lambda espressione ridotta in forma normale rappresenta quindi il **valore calcolato**, il vero e proprio risultato.

## 6.12 Teorema di Church Rosser

Il teorema afferma che il risultato non cambia al cambiare dell'ordine delle scelte da eseguire durante la semplificazione di un'espressione di lambda calcolo, in particolare:
Se esistono più percorsi di scelte, allora essi convergono necessariamente in una singola e comune espressione.

## 6.13 Funzioni di ordine superiore e currying

In lambda calculus le funzioni hanno un solo parametro, quindi per rappresentare le funzioni con più parametri è necessario effettuare un **annidamento** di $\lambda$.
Equivale a trasformare una funzione $f(X \times Y) \to Z$ in una funzione $g : X \to (Y \to Z)$.
Questa trasformazione si chiama **currying**.
Vantaggi del currying:

- **Uniformità**: ogni funzione prende un solo argomento (più semplice)
- **Applicazione parziale**: possibilità di utilizzare subito funzioni specializzate senza costrutti aggiuntivi.
- **Eleganza e minimalità**: bastano astrazione e applicazione per esprimere qualsiasi espressione.

In pratica semplifica il tutto e rende il lambda calculus molto più potente.

## 6.14 Valutazione dell'applicazione di funzione

Ecco i passi:

1. **Primo passo:** valutazione dell'espressione che definisce la funzione.
2. **Secondo passo:** due casi:
   - Si può calcolare un parametro prima di applicarlo (**Valutazione eager, Call by value**)
   - Si può applicare un parametro prima di calcolarlo (**Valutazione lazy, Call by name**)
3. Valutazione del corpo della funzione

**Haskell** utiizza lo stile lazy, mentre **OCaml** utilizza lo stile eager. Ecco alcune differenze tra call by name e call by value:

- **call by value:** argomenti valutati una sola volta, potrebbe non terminare.
- **call by name:** argomenti valutati anche più volte o anche mai, termina sempre.

## 6.15 Implementazione della ricorsione

Per rappresentare la ricorsione si usa il **combinatore** "$Y$", il quale è una funzione di ordine superiore.
Essa è una funzione speciale che, data una funzione generica $F$, costruisce un punto fisso di $F$.

- **Combinatore:** espressione che non ha variabili libere.
- **Punto fisso:** simile al concetto di caso base della ricorsione.

Vale questo:

$$YF \equiv_\beta F(YF)$$

Vediamo un esempio di implementazione della ricorsione con il concetto del fattoriale:

$$Y = \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$$
$$G = \lambda f.\lambda n.if(n = 0)then1elsen \cdot f(n-1)$$
$$F = YG = \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))G$$
$$= (\lambda x.G(xx))(\lambda x.G(xx)) = G((\lambda x.G(xx))(\lambda x.G(xx)))$$

Come si può notare si sta effettuano una ricorsione, al quale si potrà poi applicare un parametro attuale per calcolarne il risultato.

## 6.16 Variabili booleane

Ecco i valori principali in lambda calcolo dei valori booleani:

- **TRUE:** $\lambda t.\lambda f.t$
- **FALSE:** $\lambda t.\lambda f.f$

Vediamo adesso invece come funziona il **NOT**:

$$NOT = \lambda b.bTRUEFALSE$$

Se si applica TRUE o FALSE al NOT si ottiene il corrispettivo risultato.
Diamo un occhiata a **AND** e **OR**:

$$AND = \lambda b.\lambda c.bcFALSE$$

$$OR = \lambda b.\lambda c.bTRUEc$$

Vediamo come si applica l'AND:

$$\begin{aligned} AND\ TRUE\ TRUE &= (\lambda b.\lambda c.\, bc\, FALSE)\ TRUE\ TRUE \\ &\rightarrow TRUE\ TRUE\ FALSE \\ &\rightarrow TRUE \end{aligned}$$

$$\begin{aligned} AND\ TRUE\ FALSE &= (\lambda b.\lambda c.\, bc\, FALSE)\ TRUE\ FALSE \\ &\rightarrow TRUE\ FALSE\ FALSE \\ &\rightarrow FALSE \end{aligned}$$

$$\begin{aligned} AND\ FALSE\ TRUE &= (\lambda b.\lambda c.\, bc\, FALSE)\ FALSE\ TRUE \\ &\rightarrow FALSE\ TRUE\ FALSE \\ &\rightarrow FALSE \end{aligned}$$

$$\begin{aligned} AND\ FALSE\ FALSE &= (\lambda b.\lambda c.\, bc\, FALSE)\ FALSE\ FALSE \\ &\rightarrow FALSE\ FALSE\ FALSE \\ &\rightarrow FALSE \end{aligned}$$

E adesso l'OR:

$$\begin{aligned} OR\ TRUE\ TRUE &= (\lambda b.\lambda c.\, b\, TRUE\, c)\ TRUE\ TRUE \\ &\rightarrow TRUE\ TRUE\ TRUE \\ &\rightarrow TRUE \end{aligned}$$

$$\begin{aligned} OR\ TRUE\ FALSE &= (\lambda b.\lambda c.\, b\, TRUE\, c)\ TRUE\ FALSE \\ &\rightarrow TRUE\ TRUE\ FALSE \\ &\rightarrow TRUE \end{aligned}$$

$$\begin{aligned} OR\ FALSE\ TRUE &= (\lambda b.\lambda c.\, b\, TRUE\, c)\ FALSE\ TRUE \\ &\rightarrow FALSE\ TRUE\ TRUE \\ &\rightarrow TRUE \end{aligned}$$

$$\begin{aligned} OR\ FALSE\ FALSE &= (\lambda b.\lambda c.\, b\, TRUE\, c)\ FALSE\ FALSE \\ &\rightarrow FALSE\ TRUE\ FALSE \\ &\rightarrow FALSE \end{aligned}$$

## 6.17 Condizionale

La struttura corrisponde a questo:

$$[espressione][then][else]$$

Ecco come viene rappresentata:

$$IF = \lambda c.\lambda then.\lambda else.cthenelse$$

Ecco un esempio con TRUE:

$$IFTRUEe_1e_2 = (\lambda c.\lambda then.\lambda else.cthenelse)TRUEe_1e_2$$

$$= (\lambda then.\lambda else.TRUEthenelse)e_1e_2 = (\lambda else.TRUEe_1else)e_2 = TRUEe_1e_2$$

$$= (\lambda t.\lambda f.t)e_1e_2$$

$$= (\lambda f.e_1)e_2$$

$$= e_1$$

L'esempio è analogo su FALSE.

## 6.18 Numeri

Possiamo codificare i numeri naturali con una funziona che, data una funzione $s$ (successore), viene applicata $n$ volte ad una funzione $z$ (lo 0), con $n$ =numero desiderato.

$$C_0 = \lambda s.\lambda z.z$$

$$C_1 = \lambda s.\lambda z.sz$$

$$C_2 = \lambda s.\lambda z.s(sz)$$

$$C_3 = \lambda s.\lambda z.s(s(sz))$$

$$\dots$$

$$C_n = \lambda s.\lambda z.s^n z$$

Vediamo adesso come svolgere varie operazioni:

$$SUCC = \lambda n.\lambda s.\lambda z.s(nsz)$$

$$PLUS = \lambda m.\lambda n.\lambda s.\lambda z.ms(nsz) = \lambda m.\lambda n.mSUCCn$$

$$TIMES = \lambda m.\lambda n.m(PLUSn)C_0$$

In "SUCC" la $n$ segna quante volte svolgere $s$ su $z$ (quindi potrebbe anche essere $s^n$?).

# 7 Regole di inferenza

## 7.1 Sintassi

$$E =$$

$$E = TRUE \quad = FALSE = NV \quad = if\ E\ then\ E\ else\ E = succ\ E \quad = pred\ E = isZero\ E$$

$$V = TRUE \quad = FALSE = NV$$

$$NV = 0 \quad = 1 = 2 \quad = \dots$$

## 7.2 Successore

$$\frac{E \to E^I}{succ(E) \to succ(E^I)}$$

$$\frac{m = n + 1}{succ(n) \to m}$$

## 7.3 Predecessore

$$\frac{E \to E^I}{pred(E) \to pred(E^I)}$$

$$\frac{n > 0, m = n - 1}{pred(n) \to m}$$

## 7.4 IsZero

$$\frac{E \to E^I}{isZero(E) \to isZero(E^I)}$$

$$\frac{n = 0}{isZero(n) \to TRUE}$$

$$\frac{n > 0}{isZero(n) \to FALSE}$$

## 7.5 If-Else

$$if\ TRUE\ then\ E_1\ else\ E_2 \to E_1$$

$$if\ FALSE\ then\ E_1\ else\ E_2 \to E_2$$

$$\frac{E \to E^I}{if\ E\ then\ E_1\ else\ E_2 \to if\ E^I\ then\ E_1\ else\ E_2}$$

# 8 Controllo dei tipi

Il controllo dei tipi nella grammatica corrisponde ad un'associazione binaria tra **espressione** e **tipo**.

$$E : T$$

## 8.1 Tipi

Ecco i tipi usati nel type checking:

- **Nat:** numeri naturali
- **Bool:** valori booleani

Type Checking dei valori sintattici:

$$false : Bool$$

$$true : Bool$$

$$NV : Nat$$

## 8.2 Well-typed e ill-typed

Un'espressione E è **well-typed** se è ben tipata. Il contrario (debolmente tipato) indica che E è un'espressione **ill-typed**.

## 8.3 Type Safety: Correttezza

La **correttezza** (type safety o soundness) del sistema di tipo è espressa formalmente da queste proprietà:

- **Progresso:** se $E : T$ allora $E$ è un valore oppure $E \to E^I$ per una quache espressione $E^I$.
- **Conservazione:** se $E : T$ e $E \to E^I$ allora $E^I : T$.

## 8.4 Controllo su successore

$$\frac{E : Nat}{succ(E) : Nat}$$

## 8.5 Controllo su predecessore

$$\frac{E : Nat}{pred(E) : Nat}$$

## 8.6 Controllo su isZero

$$\frac{E : Nat}{isZero(E) : Bool}$$

## 8.7 Controllo su if-else

$$\frac{E : Bool, E_1 : T, E_2 : T}{if\ E then\ E_1\ else\ E_2 : T}$$

# 9 Lambda Calculus Tipato

## 9.1 L'introduzione dei tipi

Nel linguaggio del lambda calcolo appena visto possono crearsi errori di semantica tipo:

$$IF0e_1e_2 = TRUE0$$

In questo caso si può applicare TRUE a 0 e creerebbe un risultato insensato.
Per questo sono necessarie delle **regole** e l'integrazione dei **tipi**.

**Tipo:** raggruppamento di valori in base al comportamento che vorremmo avessero. Descrive come il programma deve utilizzare il tipo stesso.
Per esempio:

$$3 + 2 : Int$$

Indica che ci si aspetta che il risultato sia un intero e che l'operatore binario "+" funzioni correttamente prendendo in input 2 interi.

## 9.2 Sistema di controllo dei tipi

In pratica andiamo ad applicare un vero e proprio **sistema dei tipi**, il quale è un **metodo sintattico**, **effettivo** e **strutturale** per dimostrare l'assenza di comportamenti anomali.

- Sintattico: la struttura sintattica guida il metodo di analisi del comportamento dei programmi.
- Effettivo: si può definire un **algoritmo** che controlla i vincoli sui tipi e implementarlo in un compilatore o interprete.
- Strutturale: i tipi assegnati alle componenti del programma è definito da quello delle sue sotto-componenti.

Il controllo dei tipi può essere:

- **Statico:** compile-time
- **Dinamico:** run-time

Il controllo dei tipi è fondamentale perchè è in grado di verificare alcune proprietà del programma:

- **Segretezza e autenticità** dei protocolli di sicurezza.
- **Assenza di deadlock** in sistemi concorrenti.

## 9.3 Sintassi dei tipi

Ecco la sintassi:

$$\begin{aligned}&\tau ::= \text{ (Tipi)}\\&Bool \text{ (Tipi dei booleani)}\\&\tau \to \tau \text{ (Tipo delle funzioni)}\end{aligned}$$

Inoltre, vediamole espressioni e i valori:

$$\begin{aligned}&e ::=\\&\quad = x\\&\quad = funx : \tau = e\\&\quad = e\ e\\&\quad = Apply(e, e)\\&\quad = true\\&\quad = false\\&\quad = n = 0, 1, 2...\\&\quad = e \oplus e = \text{operazioni binarie (+,-,*,/)}\\&\quad = if\ e\ then\ e\ else\ e\end{aligned}$$

$$\begin{aligned}&V ::=\\&\quad = false\\&\quad = true \quad = fun\ x : \tau = e\end{aligned}$$

Data questo nuovo alfabeto, vediamo la sintassi come viene ridefinita:

$$Apply(fun\ x : \tau = e_1, v_2) \to e_1\{x := v_2\}$$

$$\frac{e_1 \to e^I}{Apply(e_1, e_2) \to Apply(e^I, e_2)}$$

$$\frac{e_2 \to e^I}{Apply(e_1, e_2) \to Apply(e_1, e^I)}$$

$$\frac{e \to e_4}{if\ e_1\ then\ e_2\ else\ e_3 \to if\ e_4\ then\ e_2\ else\ e_3}$$

$$if\ true\ then\ e_2\ else\ e_3 \to e_2$$

$$if\ false\ then\ e_2\ else\ e_3 \to e_3$$

## 9.4 Ambiente di tipo

L'**ambiente di tipo** (Type Enviroment) è fondamentale per effettuare il type-checking sulle **variabili**.
Si indica con $\Gamma$ ed è una **funzione** (di dominio finito) che associa nomi a tipi:

$$\Gamma = \{(x_1 : \tau_1), \ldots, (x_k : \tau_k)\}$$

Per indicare la funzione:

$$\Gamma(x_i) : \tau_i$$

Se la funzione viene applicata su una variabile che non si trova nell'ambiente, la funzione restituisce **undefined**.
Il controllo effettivo di tipo si chiama **giudizio di tipo** e si indica così:

$$\Gamma \vdash e : \tau$$

Queste regole sono applicate dal compilatore tramite analisi statica.
Vediamo una regola di inferenza molto importante sul giudizio di tipo:

$$\frac{\Gamma(x) : \tau}{\Gamma \vdash x : \tau}$$

Vediamone un altra per le funzioni:

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash funx : \tau_1 = e : \tau_1 \to \tau_2}$$

E ancora per la chiamata di funzione:

$$\frac{\Gamma \vdash e : \tau_1 \to \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash Apply(e_1, e_2) : \tau_2}$$

## 9.5 Substitution Lemma

Il lemma dice:

"I tipi sono preservati dall'operazione di sostituzione"

Se:

$$\Gamma, x : \tau_1 \vdash e : \tau$$
$$\Gamma \vdash e_1 : \tau_1$$

Allora:

$$\Gamma \vdash e\{x := e_1\} : \tau$$

## 9.6 Dichiarazioni locali

Stabiliamo che:

$$e ::= let\ x = e_1\ in\ e_2 : \tau_2$$

Significa che l'associazione "$x = e_1$ vale in $e_2$.
Regola di inferenza:

$$\frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash let\ x = e_1\ in\ e_2 : \tau_2}$$

Questo significa che il valore della variabile locale rimane nel blocco $e_2$ e non sovrascrive il tipo ritornato da tale espressione (non esce).

## 9.7 Ricorsione

L'autoapplicazione non è tipabile:

$$e ::= fix\ e$$
$$\frac{e \to e^I}{fix\ e \to e^I}$$

$$\frac{}{fix(fun\ f : \tau = e) \to e[f := (fix(fun\ f := e : \tau))]}$$

# 10 OCaml

## 10.1 Antenati

**ML (Meta Language)** fu proposto da Robin Milner nel 1972 e sviluppato negli anni '80.
Era un linguaggio funzionale fortemente tipato, ha influenzato i linguaggi successivi introducendo concetti come il polimorfismo, pattern matching e altro.
**Caml** era una versione di ML che introduce moduli e fornisce una serie di librerie di programmazione.
**OCaml** è la versione che implementa la programmazione a oggetti.

## 10.2 Come mai NON studiare OCaml (Ma invece si)

OCaml non è:

- Tra i linguaggi più popolari.
- Semplice.
- Il linguaggio del futuro.

Nonostante questo, è utile studiarlo perche molti linguaggi hanno preso concetti da OCaml e perchè è un linguaggio che mostra una programmazione funzionale realizzata al meglio. Lo useremo per implementare in modo naturale semantica statica e dinamica, ma anche per realizzare **interpreti**.

## 10.3 Esecutore di OCaml

Per utilizzare OCaml useremo:

- ocaml: interprete standard eseguibela da terminale.
- utop: interprete con interfaccia avanzata.
- Interprete online.

L'esecutore si chiama anche **interactive toplevel** e funziona cosi:

- Esegue un REPL (Read-Eval-Print Loop).
- Compila e valuta immediatamente ogni espressione, fornendo il risultato e il suo tipo.
- Il risultato può essere legato ad un nome, ottenendo una **dichiarazione**.
- Un'espressione può usare risorse precedentemente definite (variabili, funzioni...)

Vediamo un esempio di dichiarazione:

```
1  let x = 12.0 ;;
```

Adesso un'espressione:

```
1  x + 5 ;;
```

## 10.4 Overloading degli operatori e cast

OCaml non supporta l'overloading degli operatori, le operazioni devono essere **coerenti**. Per esempio per sommare un elemento float con qualcos'altro bisogna usare questo operatore (altrimenti da errore):

```
x +. 10.5 ;;
```

Non esistono quindi cast impliciti, ma solo **cast espliciti**:

```
float_of_int 3 ;; (*Restituisce 3.0*)
int_of_float 3.0 ;; (*Restituisce 3*)
```

Ecco altri cast:

- int_of_char
- int_of_string

## 10.5 Tipi di OCaml

### 10.5.1 Tipi di base

- int
- float
- bool
- char
- string
- unit: usato in casi particolari (simile a **void**)

### 10.5.2 Tipi complessi

- Tupla

```
let t = [10,"ten",10.0] ;;
let (x,y,z) = t ;; (*Destrutturazione della tupla*)

```

- **Whatever**: simile ad "any"

```
fun x -> x ;; (*Restituisce: - : 'a -> 'a = <fun>*)

```

### 10.5.3 Tipi custom

In OCaml possiamo definire nuovi tipi di dato usando il costrutto **type**.
Per esempio, possiamo assegnare dei nuovi nomi a tipi complessi:

```
type data = int*int*int ;;
```

Addirittura, possiamo definire degli **alias** per specificare un formato:

```
let controlla_data (d:data) =
  match d with
  | gg,mm,aaaa -> gg > 0 && gg <= 31
                  && mm > 0 && mm <= 12
                  && aaaa > 1980 && aaaa <= 2030 ;;
```

Nel codice sopra non sono gestiti alcuni casi che sarebbero incorretti, come il 31esimo giorno di Febbraio, ma non ci interessa, dato che doveva funzionare esclusivamente come esempio di alias.
In OCaml quindi per definire tipi nuovi si può comporre altri tipi.

## 10.6 Operazioni logiche

Ecco come si eseguono le operazioni logiche in OCaml:

- And:

```
    x && y ;;

```

- Or:

```
    x || y ;;

```

- Not:

```
    not x ;;

```

- Diverso:

```
    x <> y ;;

```

## 10.7 Operazioni sulle stringhe

Ecco un esempio di concatenazione:

```
let h = "hello " ;;
let hw = h ^ "world" ;;
```

## 10.8 Funzioni e lambda calcolo

In OCaml le funzioni sono **valori**.
Vediamo come definire una funzione in lambda calcolo:

$$\lambda x.x + 1$$

```
fun x -> x + 1 ;;
(fun x -> x + 1) 10 ;; (*Restituisce 11*)
```

$$\lambda x.\lambda y.x + y$$

```
fun x -> fun y -> x + y ;;
(*Oppure (Currying):*)
fun x y -> x + y ;;
(*Oppure:*)
fun (x, y) -> x + y ;;
```

$$\lambda f.\lambda y.fy$$

```
fun f -> fun n -> f n + 1 ;;
(*Restituisce: ('a -> int) -> int -> int*)
```

Vediamo adesso anche funzioni di ordine superiore:

$$(\lambda f.\lambda y.fy)(\lambda x.x)10$$

Le funzioni possono anche essere dichiarate con **let** e essere memorizzate:

```
let somma = fun x y -> x + y ;;
(*Oppure*)
let somma x y = x + y ;;
```

Addirittura esiste un altra keyword per dichiarare funzioni, ovvero **function**, ma questa accetta 1 solo parametro:

```
function x -> x + 1 ;;
```

```
(fun f -> fun y -> f y + 1)(fun x -> x)10 ;;
```

### 10.8.1 Applicazione parziale

Se abbiamo una funzione:

```
let somma x y -> x + y ;;
```

E si applica un solo parametro, ci ritroviamo una nuova funzione:

```
somma 10 ;; (*Restituisce: somma y = 10 + y*)
```

### 10.8.2 Encoding

In realtà esiste una trasformazione formale per convertire le espressioni di lambda calcolo in espressioni OCaml, ecco le regole:

## 10.9 Scope

Per dichiarare che un assegnamento vale solo in un'espressione particolare si usa la keyword "in", che fa valere l'assegnamento nell'espressione che la segue:

```
let x = 4 in ;;
let y = x + 2 ;; (*Qui vale*)
let z = x + 2 ;; (*Qui no!*)
```

Si può usare con le funzioni:

```
let sol a b c =
    let delta =
        b*.b -. 4.*.a*.c
    in
        (-.b +. sqrt delta) /. (2.*.a) , (-.b -. sqrt delta) /. (2.*.a)
  ;;
```

## 10.10 Costrutti

Siccome le variabili sono immutabili, è insensato svolgere cicli.
Per svolgere decisioni si usa il costrutto **if-then-else**:

```
let x = 5;;
let y = if x>10 then 1 else 0 ;;
```

## 10.11 Ricorsione

Per realizzare la ricorsione si usa la keyword "rec":

```
let rec fact n = if n<=0 then 1 else n * fact(n-1) ;;
```

Se non si usa la keyword si crea un errore, altrimenti le chiamate ricorsive non vengono interpretate come tali, bensì come valori **unbounded**.

**Mutua ricorsione:** queste due funzioni si richiamano a vicenda:

```
let rec parti n =
    match n with
        | 0 -> true
        | x -> dispari (x-1)

and dispari n =
    match n with
        | 0 -> false
        | x -> pari (x-1) ;;
```

## 10.12 Sum if true

Somma in base a condizione, vediamo un esempio applicativo:

```
sum_if_true (fun x -> x>0) 3 (-4) ;;
```

## 10.13 Type inference e polimorfismo

OCaml utilizza un type inference per stabilire il tipo delle variabili:

```
let x = 3 ;; (*Tipo inferito int*)
let id x = x ;; (*La funzione id = polimorfa*)
```

Volendo però possiamo specificarlo manualmente se vogliamo:

```
let maggiore_int (x: int) (y: int) : int = if x > y then 1 else 0 ;;
```

## 10.14 Liste

Vediamo come crearne una:

```
let lista = [] ;; (*Lista vuota*)
let lista2 = [1,2,3] ;;
```

Vediamo adesso delle operazioni sulle liste:

```
let lista1 = ["hi"] ;;
let lista2= ["ciao"] ;;

(*Concatenazione liste*)
let lista3 = lista1 @ lista2 ;;

(*Ritorno della lunghezza*)
List.length lista3 ;;

(*Ritorno della testa e della coda*)
List.hd lista3 ;;
List.tl lista3 ;;

(*Ordine inverso*)
List.rev lista3 ;;
```

### 10.14.1 Funzioni ricorsive su liste

- $[] ->$ caso base
- $x :: lis' ->$ caso induttivo

**Length:**

```
let rec length lis =
  match lis with
  | [] -> 0
  | x::lis' -> 1 + length lis' ;;
```

**Contains:**

```
let rec contains x lis =
  match lis with
  | [] -> false
  | x::lis' -> true
  | y::lis' -> contains x lis' ;;
```

**Append:**

```
let rec append l1 l2 =
  match l1 with
  | [] -> l2
  | x::l1' -> x :: (append l1' l2) ;;
```

**Inversione:**

```
let rec rev lis =
  match lis with
  | [] -> lis
  | x::lis' ->  (invert lis') @ [x] ;;
```

### 10.14.2 Funzioni higher-order su liste

Si possono utilizzare le funzioni higher-order anche sulle liste:
Vediamo un esempio e trasformiamolo con funzioni higher-order. Vediamo prima la forma iniziale:

```
let rec contiene_zero lis =
  match lis with
  | [] -> false
  | x::lis' -> if x=0 then true
               else contiene_zero lis' ;;
```

Generalizziamola e rendiamola una funzione higher-order:

```
let rec exist p lis =
  match lis with
  | [] -> false
  | x::lis' -> if p x then true
               else exist p lis' ;;
```

Si immette quindi il controllo come parametro.
Vediamo adesso un esempio di applicazione **forall**:

```
let rec forall p lis =
  match lis with
  | [] -> false
  | x::lis' -> if p x then forall p lis'
               else false ;;
```

### 10.14.3 Accumulatore

Una funzione ricorsiva sulle liste può appoggiarsi su una funzione secondaria, chiamata **accumulatore** per costruire man mano il risultato:

```
let rec rev lis =
  let rec rev_accum lis1 lis2 =
    match lis1 with
    | [] -> lis2
    | x::lis1' ->  rev_accum lis1' (x::lis2) (*Si mette x su lis2, dopo
   rev_accum lis1*);;
```

```
    in
      rev_accum lis [] ;;
```

### 10.14.4 Filter

Filtra gli elementi di una lista secondo una condizione:

```
let nums = [1, 2 ,3, 4, 5] ;;
let evens = List.filter (fun x -> x mod 2 = 0) nums ;;
```

### 10.14.5 Map

Applica una funzione ad ogni elemento della lista:

```
let nums = [1, 2 ,3, 4, 5] ;;
let result = List.map (fun x -> x * 2) nums ;;
```

### 10.14.6 Reduce (fold)

Permette di convergere piu valori in un valore solo evitando stack overflow:

```
let nums [1, 2, 3, 4, 5] ;;
let sum = List.fold_left (fun acc x -> acc + x) 0 nums ;;
```

"acc" sta per accumulator, somma tutti i numeri partendo da sinistra. Lo "0" che si vede è il **valore iniziale** da cui partire.
Ovviamente esiste anche **fold_right** oltre che **fold_left**.

## 10.15 Pattern matching

Presa un'espressione EXP, possiamo usare un costrutto simile allo switch:

```
match EXP with
| P_1 -> EXP_1
| P_2 -> EXP_2
...
| P_N -> EXP_N
```

## 10.16 Wildcard

Una wildcard è come un placeholder e si indica con "_"

```
let rec fibonacci n =
  match n with
    | 0 -> 0
    | 1 -> 1
    | _ -> fibonacci (n-1) + fibonacci (n-2) ;;
```

In questo caso è stato usato per il caso default del match.

## 10.17 Record

I **record** sono composti da più elementi, ciascuno con un proprio tipo.
Sono simili a tuple, ma possiamo dare un nome agli elementi detti **campi**.

```
type persona = {nominativo: string; eta: int} ;;
```

### 10.17.1 Dot notation

Per accedere ad un campo di un record si può usare la **dot notation**:

```
persona.nominativo ;;
```

### 10.17.2 Copia di record

Per creare un nuovo record a partire dai campi forniti da un altro record si usa **with**:

```
(*Campi di mario WITH altri nuovi campi*)
let bianca = {mario with nome = "Bianca"; eta = 13} ;;
```

### 10.17.3 immutabilità

Purtroppo i valori dei record non possono essere modificati.

```
persona.nominativo = "Mirco Rossi" ;;
```

Questo da errore.

## 10.18 Variant

Un tipo **variant** (o tipo **unione**) prende solo uno tra i valori specificati.

```
let numero_testo =
    | Txt of string
    | Num of int ;;

let x = Txt "34" ;;
let y = Num 26 ;;
```

In questo caso, la variant può assumere un valore di stringa o un valore numerico.
I variant possono essere usati per controllare il tipo di una variabile:

```
match x with
|  Txt s -> int_of_string s
|  Num n -> n ;;
```

### 10.18.1 Enum

L'enum in OCaml è rappresentao come un tipo particolare di variant:

```
type giorno = Lun | Mar | Mer | Gio | Ven | Sab | Dom ;;
```

### 10.18.2 Numeri naturali estesi

Il variant può rappresentare un estensione del tipo dei valori interi:

```
type int_ext =
| Num of int
| Nan
| Plus_inf
| Minus_inf ;;

let sum (x: int_ext) (y: int_ext) =
  match x y with
  | NaN,_ | _,NaN -> NaN
  | Plus_inf,Minus_inf | Minus_inf,Plus_inf -> NaN
  | Plus_inf,_ | _Plus_inf -> Plus_inf
  | Minus_inf,_ | _,Minus_inf -> Minus_inf
  | Num n1, Num n2 -> n1 + n2 ;;
```

## 10.19 Tipi ricorsivi

I **tipi ricorsivi** sono tipi variant definiti in termini di se stessi.
Servono per rappresentare strutture di dimensione arbitraria (anche infinite), come liste, alberi e altro.

```
type lista_di_int =
     | Nil
     | Elem of int * lista_di_int ;;

let lst = Elem (3 , Elem (4, Elem (6, Nil))) ;;
```

Questo tipo ricorsivo rappresenta una lista di interi, diversa da una semplice lista di interi rappresentata nel modo classico.
Questa rappresentazione è utile per capire come funzionano alcuni metodi sulle liste.
**Aggiunta in testa:**

```
let cons x lis =
    Elem (x,lis) ;;

cons 5 lst ;;
```

### 10.19.1 Alberi

Vediamo come realizzarli con i tipi ricorsivi:

```
(*Alberi binari*)
type albero_bin =
     | Nodo of int * albero_bin * albero_bin
     | Foglia of int ;;
```

In questo albero o un nodo ha due figli, o è foglia. Questa è una rappresentazione semplice, per fare in modo che un figlio possa essere nullo possiamo usare un tipo di appoggio (variant) che può assumere anche il valore nullo.

Vediamo alcuni metodi:

```
(*Visita anticipata, albero binario*)
let rec visita_ant a =
  match a with
  | Foglia v -> [v]
  | Nodo (v,sx,dx) -> v::((visita_ant sx)@(visita_ant dx)) ;;

(*Somma dei nodi, albero binario*)
let rec somma_albero a =
  match a with
  | Foglia v -> v
  | Nodo (v,sx,dx) -> v + somma_albero sx + somma_albero dx ;;
```

### 10.19.2 Abstract Syntax Tree

Coi tipi ricorsivi possiamo realizzare anche gli alberi di sintassi astratta, utili per rappresentare un analisi sintattica.
Prendiamo per esempio:

$$\text{if } (x > y) \text{ then } max = x; \text{ else } max = y;$$

Questo può essere rappresentato da un albero radicato in if con figli uguali agli operatori, i quali come figli hanno i termini delle operazioni.
Vediamo come realizzare per esempio un AST delle operazioni aritmetiche base:

```
let op = Add | Sub | Mul | Div ;;
let exp =
  | Val of int
  | Op of op * exp * exp
  | UMin of exp ;;
```

## 10.20 Programmazione imperativa (introduzione)

Abbiamo lavorato fin'ora con la programmazione funzionale, andiamo adesso a vedere la programmazione imperativa.
Adesso tutto ha una causa e un effetto, le funzioni non servono piu solo a calcolare, ma possono essere influenzate da membri esterni.

### 10.20.1 Nuova sintassi degli array

**Definizione:**

```
let a = [|3,2,1|] ;;
(*Oppure*)
let a = Array.make 3 None ;;
```

**Lunghezza:**

```
let n = Array.length a ;;
```

**Accedere ad un elemento:**

```
let e = a.(i) ;;
```

**Modifica di un elemento:**

```
a.(i) <- 6 ;;
```

Questa è la prima volta in OCaml che possiamo modificare qualcosa!

### 10.20.2 Record mutabili

Normalmente non si potrebbe, ma adesso i record sono mutabili!
Per farlo, bisogna rendere un record mutabile tramite la keyword **mutable**, il quale rende possibile la modifica di un campo senza dover creare un record secondario:

```
type Persona = {
  nominativo: string;
  mutable eta: int
} ;;
```

### 10.20.3 Riferimenti (refs)

Si possono definire anche singole variabili mutabili:

```
let x = ref 12 ;;
```

Questa variabile contiene un **riferimento** al valore in memoria, accedibile e modificabile tramite "contents":

```
x.contents ;;
x.contents <- 14 ;;
```

Esiste una sintassi più semplice:

```
(*Equivale a x.contents*)
!x ;;
(*Equivale a x.contents <- 14*)
x := 14 ;;
(*Equivale a x.contents <- x.contents + 1*)
x := !x + 1 ;;
```

### 10.20.4 Sequenze di espressioni e cicli

Una sequenza di espressioni si crea cosi:

```
x := !x+1 ; y := !y+1 ; (!x,!y) ;;
```

Vediamo adesso il ciclo **for:**

```
for i = 0 to (Array.length arr)-1 do
  ...
done ;
```

Adesso il ciclo **while**:

```
let x = ref 10 in
while !x > 0 do
  ...
done ;
```

### 10.20.5 Eccezioni

Ecco come si creano eccezioni custom:

```
exception Lista_Vuota ;;
exception Stringa_Errata of string ;;
```

Ecco come si lanciano:

```
raise Lista_Vuota ;;
raise (Stringa_Errata "test") ;;
```

Esiste un costrutto che permette di svolgere delle funzioni in modo che si attribuisca un risultato anche alle eccezioni:

```
try
  tutti_pos lis
with
| Lista_Vuota -> true ;;
```

In particolare, esiste un'eccezione astratta chiamata **Failure** che possiamo sollevare senza usare "raise":

```
failwith("Messaggio di errore") ;;
```

Un'altra eccezione viene sollevata quando **assert** non viene soddisfatta.

## 10.21 Encoding da lambda calcolo a OCaml

Ecco le regole sintattiche di trasformazione:

$$\begin{aligned} [[n]] &= n \\ [[x]] &= x \\ [[T^I op T^{II}]] &= [[T^I]] op [[T^{II}]] \\ [[\lambda x.T^I]] &= fun\ x\ ->\ [[T^I]] \\ [[T^I T^{II}]] &= [[T^I]][[T^{II}]] \end{aligned}$$

In pratica è molto semplice, queste trasformazioni lasciano tutto inalterato e sostituiscono la denotazione con $\lambda$.

# 11 Macchine astratte e interpreti

Per eseguire dei programmi scritti in un certo linguaggio, bisogna convertirlo in linguaggio macchina, per farlo esistono 3 approcci:

- **Compilazione:** traduzione dell'intero codice e creazione di un file eseguibile.
- **Interpretazione:** usa un procedimento stile fetch-decode-execute, riga per riga.
- **Misto**: si compila e poi si interpreta l'eseguibile (che si chiama **bytecode**) (Java, Python, Typescript (che viene compilato in JS)).

## 11.1 Macchine astratte

Una **macchina astratta** è un sistema virtuale che rappresenta il comportamento di una macchina fisica individuando:

- L'insieme delle risorse necessarie per l'esecuzione di programmi
- Un insieme di istruzioni specificatamente progettato per operare con queste risorse

In pratica è una collezione di **algoritmi e strutture dati** in grado di memorizzare ed eseguire programmi.
Della macchina astratta fanno parte:

- Interprete
- Memoria
- Controllo
- Operazioni "primitive"

Una macchina astratta, se implementata, diventa una macchina fisica, in cui si hanno:

- **CPU:** contiene interprete, controllo e operazioni primitive
- **RAM:** contiene la memoria.

Nel lato software (di programmazione) ci sono invece altre corrispondenze:

- **Interprete:** converte istruzioni in un formato diverso, come la funzione **eval**.
- **Controllo:** istruzioni di controllo della memoria per esempio.
- **Memoria:** definizioni di strutture dati.
- **Operazioni primitive:** funzioni.

### 11.1.1 Interprete

L'interprete prende in input un programma da eseguire, tipicamente l'albero di sintassi astratta del programma, e lo esegue ispezionandone la struttura per vedere cosa va fatto. In particolare, questo è il ciclo di operazioni che l'interprete svolge:

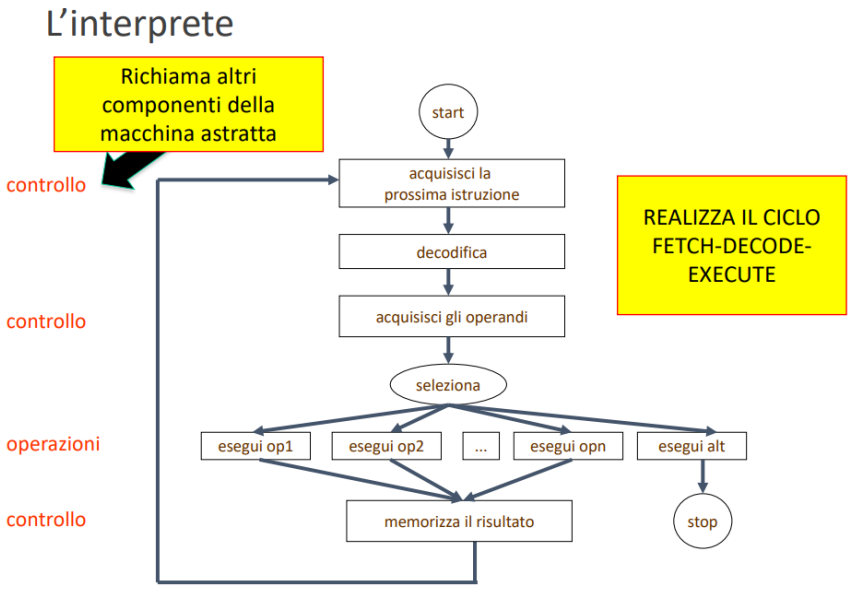



Figure 1: Interprete: ciclo fetch-decode-execute

### 11.1.2 Componente di controllo

Il componente di controllo della macchina astratta è una collezione di strutture dati e algoritmi che serve per:

- acquisire la prossima istruzione
- acquisire gli operandi e memorizzare i risultati
- gestire le chiamate e ritorni da sottoprogrammi
- gestire i thread
- gestire dinamicamente la memoria
- ...

### 11.1.3 Memoria

Il componente di memoria si occupa della memorizzazione dei dati necessari allo svolgimento del programma.

### 11.1.4 Operazioni primitive

Una collezione di strutture dati e algoritmi per svolgere ogni singola operazione prevista dal linguaggio.
Lo svolgimento di tali operazioni è preceduto dal type checking.

## 11.2 Run-Time Support

Il **Run-Time Support** offre dei riferimenti ai servizi del **Run-Time Enviroment** fornito dal linguaggio di programmazione con cui abbiamo scritto il codice sorgente, e serve per diverse cose al momento di esecuzione:

- Allocazione e liberazione della memoria
- Garbage collection
- Gestire eccezioni ed errori
- Type checking
- I/O (file, tastiera...)
- Chiamate di funzione e stack

La **macchina opsite** da sola possiede solo le istruzioni e non si può considerare come esecutore assoluto del programma, va incluso anche il **RTS**:

$$M_O = \text{ macchina opsite}$$

$$M_I = \text{ macchina intermedia}$$

$$M_I = M_O + RTS$$

Il codice prodotto dalla compilazione di un programma quindi è una sequenza di istruzioni macchina con eventuali chiamate all'RTS.

## 11.3 Compilatore

Il compilatore prende in input il codice sorgente, analizza la sintassi e la semantica e poi genera il codice in linguaggio macchina.
Le responsabilità del compilatore sono quindi quelle di garantire una valida struttura sintattica e generare un codice compatibile con il SO della macchina ospite.

### 11.3.1 Struttura del compilatore

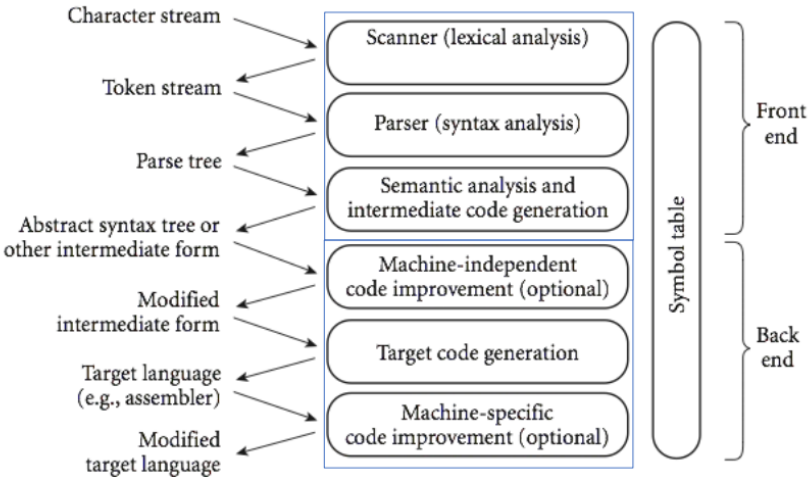



Figure 2: Struttura del compilatore

**Frontend:**

- **Scanner:** trasforma il programma sorgente nel lessico (token). Per esempio:

```
1     if (x >= y) y = 42 ;
2
```

  Se dato in pasto allo scanner, il token risultante è:

```
1     [IF][LPAREN][ID(x)][GEQ][ID(y)][RPAREN][ID(y)]
2     [BECOMES][INT(42)][SCOLON]
3
```

- **Parser:** legge il token e genera il codice intermedio (Intermediate Representation - IR). Tale risultato si trova solitamente nel formato di albero di sintassi astratta.

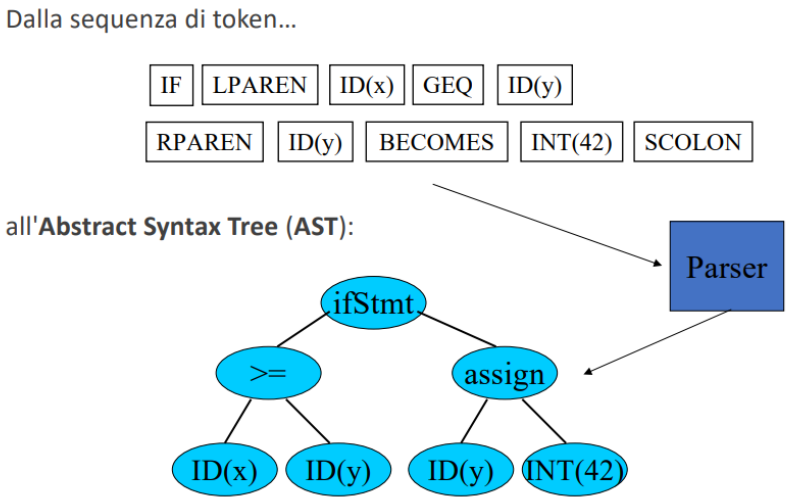



Figure 3: Parsing

Dopo tali operazioni vengono effettuate queste operazioni:

- **Type Checking**
- **Analisi statiche**
- **Eventuali ottimizzazioni**

Come fa l'interprete a capire di che tipo sono i dati? Dati di tipo diverso potrebbero avere la stessa rappresentazione binaria, per questo esistono dei campi aggiuntivi chiamati **descrittori** che indicano il tipo di una variabile in memoria.
Questo problema avviene solo nei linguaggi con type checking dinamico e provoca un maggiore consumo di memoria e maggiore tempo di controllo.
**Backend:**

- **Traduzione in linguaggio macchina**: o della macchina intermedia (Assembly per esempio).

# 12 Sviluppo di interpreti

I componenti principali dell'interprete sono:

- Scanner
- Parser
- Type Checker
- Interpreter / Executor

L'interprete funziona come il frontend del compilatore, ma l'interprete esegue le operazioni una dopo l'altra, non tutte insieme.

## 12.1 Realizzazione d'esempio

Adesso cercheremo di realizzare un interprete per le espressioni aritmetiche.

### 12.1.1 Sintassi

$$Exp ::= n|Exp\ op\ Exp|(Exp)$$

$$op ::= +|-|*|/$$

Per trasformarlo in codice, si usano i tipi variant e unione:

```
type op = Add | Sub | Mul | Div ;;
type exp =
  | Val of int
  | Op of op*exp*exp ;;
```

Proponiamo la funzione che ritrasforma l'albero delle espressioni in formato testuale:

```
let rec to_string e =
  let symbol o =
    match o with | Add -> "+" | Sub -> "-" | Mul -> "*" | Div -> "/"
  in
  match e with
  | Val n -> string_of_int n
  | Op (o,e1,e2) -> "("^ (to_string e1) ^ (symbol o) ^ (to_string e2) ^")
   " ;;
```

### 12.1.2 Definizione scanner e token

Definiamo ogni simbolo che può apparire con un token:

```
type token =
  | Tkn_NUM of int (* numero n *)
  | Tkn_OP of string (* operatore + - * / *)
  | Tkn_LPAR (* simbolo ( *)
  | Tkn_RPAR (* simbolo ) *)
  | Tkn_END;; (* fine dell'espressione *)
```

Lo scanner effettua la prima trasformazione, controllando che non ci siano simboli non previsti.
Lo scanner non effettua ancora un controllo grammaticale, per esempio l'ordine corretto delle parentesi.
Facciamo in modo che i simboli non previsti fanno lanciare un'eccezione.

```
  exception ParseError of string*string ;;
```

Vediamo la funzione scanner:

```
  (* scanner *)
let tokenize s =
  (* funzione che scandisce ricorsivamente s,
  dove pos e' la posizione del carattere corrente *)
  let rec tokenize_rec s pos =
    if pos=String.length s then [Tkn_END] (* caso base: fine lista *)
    else
      let c = String.sub s pos 1 (* estrae il carattere corrente *)
      in
      (* si richiama ricorsivamente prima di gestire il carattere corrente,
      per avere la lista dei token successivi, da completare
      con il token corrispondente al carattere corrente *)
      let tokens = tokenize_rec s (pos+1)
      in
        (* trasforma il carattere corrente in un token *)
        match c with
        | " " -> tokens
        | "(" -> Tkn_LPAR::tokens
        | ")" -> Tkn_RPAR::tokens
        | "+" | "-" | "*" | "/" -> (Tkn_OP c)::tokens
        | "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ->
          (* accorpa cifre consecutive *)
          (match tokens with
          | Tkn_NUM n::tokens' ->
          Tkn_NUM (int_of_string (c^(string_of_int n)))::tokens'
          | _ -> Tkn_NUM (int_of_string c)::tokens
          )
        | _ -> raise (ParseError ("Tokenizer","unknown symbol: "^c))
in
  tokenize_rec s 0 ;;
```

Esempio d'uso:

```
  let t1 = tokenize "(34 + 41) - (2223 * 2)" ;;
```

Ecco altre funzioni di utilità:

```
  let string_of_token t =
    match t with
    | Tkn_NUM n -> "Tkn_NUM "^(string_of_int n)
    | Tkn_OP s -> "Tkn_OP "^s
    | Tkn_LPAR -> "Tkn_LPAR"
    | Tkn_RPAR -> "Tkn_RPAR"
    | Tkn_END -> "Tkn_END" ;;
```

```
let print_token t =
  print_endline (string_of_token t);;

let print_tokenlist tl =
  List.iter (fun t -> (print_token t)) tl;;
```

## 12.2 Definizione parser

Il parser controlla che l'espressione sia sintatticamente corretta e che appartenga al linguaggio e genera l'AST.
Come realizzare un parser:

- La grammatica non deve essere ambigua
- se il linguaggio è molto semplice, si usa un parser a **discesa ricorsiva**
- se il linguaggio non è molto semplice, si usa un **parser generator** (software che genera il parser a partire dalla grammatica).

  Flex/Bison per C

  PEG.js, ATNLS per JS

  ocamllex/ocamlyacc o Menhir per OCaml

  ATNLR4 o JavaCC per Java

Trasformiamo quindi la grammatica di prima, che era ambigua e poteva presentare errori semantici, in una grammatica non ambigua:

$$Exp ::= Term + Exp | Term - Exp | Term$$

$$Term ::= Factor * Term | Factor / Term | Factor$$

$$Factor ::= n | (Exp)$$

Questa sintassi raggruppa a destra le ambiguità.

Implementiamo un parser a discesa ricorsiva:

- Una funzione per ogni categoria sintattica
- Le funzioni sono mutuamente ricorsive
- La scelta della regola da applicare avviene tramite l'analisi del token successivo (lookahead)
- Ogni chiamata restituisce un nodo dell'AST

```
(* parser *)
let parse s =
  (* usiamo un riferimento mutabile per scandire la lista dei token
     ottenuta da tokenize *)
  let tokens = ref (tokenize s) in
```

```
6   (* restituisce il primo token senza rimuoverlo *)
7   let lookahead () = match !tokens with
8     | [] -> raise (ParseError ("Parser","lookahead error"))
9     | t::_ -> t
10  in

12  (* elimina il primo token *)
13  let consume () = match !tokens with
14    | [] -> raise (ParseError ("Parser","consume error"))
15    | t::tkns -> tokens := tkns
16  in

18  (* funzioni mutuamente ricorsive che seguono dalla grammatica *)
19  (* Exp ::= Term [ + Exp | - Exp ] *)
20  let rec exp () =
21    let t1 = term() in
22    match lookahead () with
23    | Tkn_OP "+" -> consume(); Op (Add,t1,exp())
24    | Tkn_OP "-" -> consume(); Op (Sub,t1,exp())
25    | _ -> t1

27  (* Term ::= Factor [ * Term | / Term ] *)
28  and term () =
29    let f1 = factor() in
30    match lookahead() with
31    | Tkn_OP "*" -> consume(); Op (Mul,f1,term())
32    | Tkn_OP "/" -> consume(); Op (Div,f1,term())
33    | _ -> f1

35  (* Factor ::= n | ( Exp ) *)
36  and factor () =
37    match lookahead() with
38    | Tkn_NUM n -> consume(); Val n
39    | Tkn_LPAR -> consume(); let e = exp() in
40      (match lookahead() with
41      | Tkn_RPAR -> consume(); e
42      | _ -> raise (ParseError ("Parser","RPAR error"))
43      )
44  | _ -> raise (ParseError ("Parser","NUM/LPAR error"))

46  (* Si comincia chiamando exp che fa tutto il lavoro e restituisce la
     radice
47  dell'AST *)
48  in
49    let ast = exp() in
50    (* controlliamo che al termine sia rimasto solo Tkn_END *)
51    match lookahead() with
52    | Tkn_END -> ast
53    | x -> print_tokenlist !tokens; raise (ParseError ("Parser","parse
     error"));;
```

L'implementazione dei parser non è semplice:

- Può essere difficile realizzare una grammatica non ambigua
- Con la discesa ricorsiva è facile imbattersi in ricorsioni infinite

  con la grammatica usata adesso non ci sono problemi perche una funzione richiama se stessa se trova un simbolo, altrimenti no, se fosse venuta prima la chiamata ricorsiva che la chiamata mutuamente ricorsiva allora si sarebbe creata una ricorsione infinita.

## 12.3 Definizione interpreter

Per definire l'interprete, partiamo dalle definizioni della semantica.

### 12.3.1 Structural Operational Semantics (SOS)

La definizione di una relazione di transizione per descrivere il comportamento dei programmi (o espressioni) scritti in un certo linguaggio segue solitamente l'approccio della Structural Operational Semantics (SOS), o Semantica Strutturale Oprazionale:

- **Semantics:** è una descrizione del significato del linguaggio, nei linguaggi di programmazione tale significato è dato dal comportamento del programma.
- **Operational:** descrive il comportamento dei programmi tramite una relazione di transizione che cattura le operazioni che vengono svolte passo-passo.
- **Structural:** la relazione di transizione è definita usando regole di inferenza basate sulla struttura sintattica.

Esistono due approcci per realizzare una semantica nello stile SOS:

- **Approccio small-step:** specifica la valutazione delle espressioni un passo alla volta, ogni regola descrive una singola transizione che riduce l'espressione di un passo.
- **Approccio big-step:** specifica la valutazione in passi piuttosto grandi: data un'espressione, le sue regole di valutazione producono direttamente un valore.

Per esempio, se abbiamo un'espressione aritmetica, una semantica small-step svolge passo passo la semplificazione, una semantica big-step invece restituisce direttamente il valore appoggiandosi su regole di inferenza per effettuare il calcolo.

### 12.3.2 Realizzazione interprete big-step

Procediamo con l'approccio big-step (più facile):

$$n \rightarrow_{bs} n$$

$$\frac{E_1 \rightarrow_{bs} n_1, E_2 \rightarrow_{bs} n_2, n_1 \ op \ n_2 = n}{E_1 \ op \ E_2 \rightarrow_{bs} n}$$

```
  (* interprete big-step *)
let rec eval e =
  match e with
  | Val n -> Val n
  | Op (op,e1,e2) ->
    (* chiamate ricorsive che calcolano le derivazioni per e1 ed e2 *)
    match (eval e1, eval e2) with
    | (Val n1, Val n2) -> (match op with (* calcola n1 op n2 *)
                          | Add -> Val (n1+n2)
                          | Sub -> Val (n1-n2)
                          | Mul -> Val (n1*n2)
                          | Div -> Val (n1/n2)
                          )
    (* caso ( inutile ) aggiunto solo per rendere esaustivo il pattern
     matching!*)
    | _ -> failwith "Errore impossibile che si verifichi" ;;
```

### 12.3.3 Realizzazione interprete small-step

Vediamo adesso lo stesso interprete ma in small-step:

```
  (* interprete small-step *)
  let rec eval_ss e =
    match e with
    | Val n -> Val n
    | Op (op,e1,e2) ->
      (match (e1,e2) with
      (* se gli operandi sono entrambi valori, calcola n1 op n2 *)
      | (Val n1,Val n2) -> Val (match op with
                                | Add -> n1 + n2 | Sub -> n1 - n2
                                | Mul -> n1 * n2 | Div -> n1 / n2 )
      (* se e1 puo' fare un passo ( cioe' se e' un Op e non un valore ) *)
      | (Op (_,_,_),_) -> Op (op,(eval_ss e1),e2)
      (* altrimenti, se e1 e' un valore ma e2 puo' fare un passo *)
      | (Val _,Op (_,_,_)) -> Op (op,e1,(eval_ss e2))
      ) ;;
```

### 12.3.4 Dalla SOS al codice

Sistematicamente, ecco come tradurre la SOS a codice:

- SI usa il pattern matching per considerare i vari tipi di nodo AST
- Ogni tipo di nodo corrisponde ad un costrutto sintattico definito dalla grammatica
- Per ogni caso del pattern matching si identificano le regole della semantica che descrivono quel costrutto.
- Si verificano le precondizioni delle varie regole tramite chiamate ricorsive all'interprete stesso (per regole che non sono assiomi)

- Quando si trova una regola le cui precondizioni sono verificate, si calcola il risultato della transizione.

### 12.3.5 Interprete risultante

Mettiamo adesso tutto insieme, Scanner, Parser e Interpreter:

```
(* con inteprete big-step *)
let exec1 s = eval (parse s) ;;

(* con inteprete small-step *)
let exec2 s =
  let rec eval_rec ast = (* chiusura transitiva *)
    match ast with
    | Val n -> Val n
    | _ -> eval_rec ( eval_ss ast )
in
  eval_rec (parse s) ;;
```

## 12.4 Interprete per lamba calcolo

### 12.4.1 Sintassi del lambda calcolo

$$e ::= x|\lambda x.e|e\ e$$

Definiamo il tipo degli identificatori e dell'AST delle espressioni:

```
type id = string

type exp =
| Var of int
| Lam of id * exp
| App of exp * exp
```

### 12.4.2 Semantica del lambda calcolo

$$(\lambda x.e_1)e_2 \to e_1 x := e_2$$

$$\frac{e_1 \to e_1^I}{e_1 e_2 \to e_1^I e_2}$$

$$\frac{e_2 \to e_2^I}{e_1 e_2 \to e_1 e_2^I}$$

$$\frac{e \to e^I}{\lambda x.e \to \lambda x.e^I}$$

L'implementazione della capture-avoiding substitution:

$$x\{x := e\} \equiv e$$

$$y\{x := e\} \equiv y \quad \text{se } y \neq x$$

$$(e_1 e_2)\{x := e\} \equiv (e_1\{x := e\})(e_2\{x := e\})$$

$$(\lambda x.e_1)\{x := e\} \equiv (\lambda x.e_1)$$
$$(\lambda y.e_1)\{x := e\} \equiv \lambda y.(e_1\{x := e\})$$
$$(SE\ y \neq x \text{ e } y \notin FV(e))$$
$$(\lambda y.e_1)\{x := e\} \equiv \lambda z.((e_1\{y := z\})\{x := e\})$$
$$(SE\ y \neq x \text{ e } y \in FV(e) \text{ e } z \text{ fresca})$$

### 12.4.3 Implementazione di FV(e)

$$FV(x) = x$$
$$FV(\lambda x.e) = FV(e)\ x$$
$$FV(e_1 e_2) = FV(e_1) \cup FV(e_2)$$

Ecco un esempio di funzione che calcola ricorsivamente l'insieme delle variabili libere dell'espressione:

```
(* calcola le variabili libere ( non legate ) in e *)
let rec fvs e =
  match e with
  |  Var x -> [x]
  | Lam (x,e) -> List.filter (fun y -> x <> y) (fvs e)
  | App (e1,e2) -> (fvs e1) @ (fvs e2)
;;
```

## 12.5 Implementazione dell'interprete di un nucleo di linguaggio funzionale

Finora abbiamo valutato espressioni come:

$$3 + 5 * 2$$

Per le quali l'interprete può calcolarne il valore tramite AST.

**Ambiente di un interprete:** permette all'interprete di determinare a cosa si riferiscono i nomi presenti in un programma.
In ogni momento dell'eseczuone mantiene le associazioni tra nomi e entità a cui sono legati.

# 13 Altro

Il tipo dei comandi in OCaml è **unit**.

# 14 Esercizi

## 14.1 Esercizio 1

Sottolineare le variabili libere nelle seguenti espressioni:

1. $(\lambda x.x)(\lambda y.xx)(\lambda y.y)y$
2. $(\lambda x.xk)((\lambda x.\lambda y.xy)(\lambda z.(\lambda x.z)y))$
3. $x(\lambda x.(\lambda y.yy)x)y$

**Soluzione (non controllata):**

1. $(\lambda x.x)(\lambda y.\underline{xx})(\lambda y.y)\underline{y}$
2. $(\lambda x.x\underline{k})((\lambda x.\lambda y.xy)(\lambda z.(\lambda x.z)\underline{y}))$
3. $\underline{x}(\lambda x.(\lambda y.yy)x)\underline{y}$

## 14.2 Esercizio 2

Date le seguenti $\lambda$-espressioni, individuare le sottoespressioni riducibili tramite la $\beta$-riduzione (i "redex"):

1. $(\lambda x.x)((\lambda x.x)(\lambda z.(\lambda x.x)z))$
2. $(\lambda x.x)(\lambda x.xx)(\lambda y.y)$
3. $x((\lambda x.(\lambda y.yy)x)y)$

**Soluzione** Soluzione non presente, complicata da rappresentare in LaTeX.

## 14.3 Esercizio 3

Calcolare il risultato delle seguenti sostituzioni (capture avoiding):

1. $(\lambda y.xy)\{x := y\}$
2. $(\lambda y.xy)\{x := \lambda y.y\}$
3. $(\lambda y.xy)\{x := \lambda x.xy\}$

**Soluzione**

1. $(\lambda y.xy)\{x := y\} \equiv_\alpha (\lambda z.xz)\{x := y\} = \lambda z.yz$
2. $(\lambda y.xy)\{x := \lambda y.y\} = \lambda y.(\lambda y.y)y$
3. $(\lambda y.xy)\{x := \lambda x.xy\} \equiv_\alpha (\lambda z.xz)\{x := \lambda x.xy\} = \lambda z.(\lambda x.xy)z$

## 14.4 Esercizio 4

Applicare la $\beta$-riduzione alle seguenti espressioni fino a raggiungere un'espressione in forma normale (cioè, non ulteriormente riducibile) o ad accorgersi che la $\beta$-riduzione non termina. Ad ogni passo, sottolineare il "redex" considerato per l'applicazione della $\beta$-riduzione. In caso di possibili soluzioni alternative, svolgere l'esercizio più volte esplorando tutte le possibilità.

1. $(\lambda x.\lambda y.xy)(\lambda z.zy)y$
2. $(\lambda x.\lambda y.xy)(\lambda x.yy)k$
3. $(\lambda x.(\lambda y.xy)(\lambda k.k))(xy)$
4. $(\lambda x.y)((\lambda y.yyy)(\lambda x.xxx))$
5. $(\lambda z.((\lambda x.\lambda y.xy)z)z)(\lambda z.zz)$

**Soluzione**

## 14.5 Esercizio 5

Ripetere l'esercizio 1.4 applicando le strategie di riduzione Call-by-value e Call-by-name.

**Soluzione**

## 14.6 Esercizio 6

Dimostrare la $\beta$-equivalenza delle seguenti coppie di $\lambda$-espressioni. La $\beta$-equivalenza vale per ogni coppia ed è richiesto di dimostrarlo. Nella dimostrazione, cercare di svolgere il numero minore possibile di passi di $\beta$-riduzione:

1. $(\lambda x.x)(\lambda y.y)z \equiv_\beta (\lambda x.(\lambda k.k))wz$
2. $(\lambda x.x)(\lambda y.y) \equiv_\beta (\lambda y.y)(\lambda x.x)$
3. $(\lambda x.\lambda y.xy)y \equiv_\beta (\lambda x.(\lambda x.x)(\lambda z.yz))k$

**Soluzione**